Source: docs\technical\architecture.md | Export: 11 November 2026

> **Current shipping status (11 November 2026):** `DISTANCE_DELIVERY_ENABLED=false` by default. New checkout uses the existing saved flat shipping configuration, before coupons/points, with free shipping strictly above its saved threshold. Google Routes, dispatch coordinates, slabs and distance quotes are inactive. Historical order charges are unchanged. Distance-specific material below describes a retained, inactive implementation until a future explicitly approved re-enable. See current flat shipping reference.

Application architecture

For developers. Baseline: 11 November 2026, local HEAD `405716c` plus the existing delivery changes. Provider and hosted deployment state: **Pending confirmation**.

Frontend composition

`src/main.tsx` boots React. `src/App.tsx` separates Admin from the storefront, composes customer authentication/content/campaign providers and selects customer views. Navigation uses history/popstate and legacy hash normalization, not a routing-library migration. Admin is lazy-loaded behind `src/admin/AdminAccess.tsx` and uses `AdminPanel.tsx`/`AdminLayout.tsx`.

Customer Firebase Auth restores the shared session; customer and Admin routes share Auth initialization. Operators must account for shared sign-out rather than assume separate independent logins. Customer profile, shopping, addresses and order modules handle their own authenticated API contracts and stale-request/account-change protection.

Responsibility	Source
Firebase browser configuration and clients	`src/services/firebase/`
Public products/discovery	`src/products/hooks.ts`, `src/products/PublicProduct.tsx`, `src/views/ShopView.tsx`
Account shopping and checkout source/drafts	`src/shopping/`, `src/checkout/source.ts`
Address/review state	`src/delivery/useAddressBook.ts`, `src/delivery/useCheckoutReview.ts`
COD/online submit and recovery	`src/orders/useCodOrder.ts`, `src/payments/useOnlinePayment.ts`
Loading/retry/offline presentation	`src/loading/`, existing loading guide
Shared overlay focus/scroll handling	`src/components/modalLayer.ts`
Admin freshness	`src/admin/dashboard/resource.tsx`, permission/session modules

Do not treat legacy `storeService` compatibility state or fixture modules as the authoritative commerce backend. Trace mounted components and API calls. Importing a legacy component does not prove it is reachable; the standalone `AdminReports` component is not mounted by the inspected `AdminPanel`.

Backend and trust boundaries

`api/catalog.ts` lazily initializes Firebase, Cloudinary and `catalogService` inside the handler's error boundary. `server/handler.ts` serves public reads and authenticated action-based writes. `server/catalog.ts` composes catalog, delivery, orders, shopping, settings, content, business, directory, campaigns, coupons and loyalty services.

`api/payments/[action].ts` exports Web handlers so original webhook bytes reach signature validation. `server/payments/http.ts` applies method, request/origin, HTTPS, environment and worker authorization checks before dispatching. Payments reuse the existing order-reservation service, rather than maintain another order system.

`server/firebase.ts` fixes project `parinaya-silks` and database `(default)`, validates emulator isolation and uses server credentials. The browser never receives Admin credentials, Razorpay secrets or the Google Routes key. The Admin SDK bypasses Firestore rules, so service authorization/validation remains essential.

I/O and state

Public content uses server projections; private settings remain private. Direct Firebase listeners are scoped by rules, including owned shopping/address data and the revision-only shipping signal. Admin resource refresh uses the existing pulse/polling mechanism. External calls happen outside order transactions where required; accepted transactional snapshots remain authoritative.

Signed Cloudinary upload tickets and subsequent server asset verification bind media to its intended use. Google Routes prepares temporary quotes; final order placement revalidates them in the same transaction as discounts and reservations. The payment worker reconciles accepted events and attempts with the provider under mode/key isolation.

Appearance, SEO and analytics

Storefront theme is an Admin-managed projection; it is not an Admin theme. Preserve Lora and supplied artwork. Product images must not be tinted to simulate a canvas colour.

`src/content/Seo.tsx` owns document metadata/product schema and commits analytics after title selection. `server/seo.ts` generates the sitemap through a Vercel rewrite. Both currently hard-code the older `https://parinayasilks.vercel.app` origin: custom-domain SEO parity is **Pending confirmation / separate application review**, not resolved by these documents.

GA4 is a single optional adapter with explicit opt-in, production-domain/environment checks, privacy-signal handling and sanitized manual `page_view` events. Account/order identifiers and query strings are removed from transmitted page locations. Admin, Preview, local and automated contexts are excluded. Enhanced Measurement must follow the existing GA4 guide; ecommerce measurement is not implemented by the basic page-view tag.

Development boundaries

Fixtures and provider doubles belong only in isolated test harnesses. The new route calculator rejects injected doubles unless both exact loopback emulator endpoints are present and the process is not hosted on Vercel. Normal development can still point at real Firebase; see setup before using it.

Detailed contracts belong in API reference, Firestore/security, commerce transactions and delivery architecture, rather than being duplicated here.